

1

Axioms for the Real Numbers and the Positive Integers

In this book we have assumed an explicit set of axioms for the set of real numbers and for the set of positive integers. In this appendix we will list these axioms and we will illustrate how basic facts, also used without proof in the text, can be derived using them.

A1.1 Axioms for Real Numbers

The standard axioms for real numbers include both the **field** (or **algebraic**) **axioms**, used to specify rules for basic arithmetic operations, and the **order axioms**, used to specify properties of the ordering of real numbers.

THE FIELD AXIOMS We begin with the field axioms. As usual, we denote the sum and product of two real numbers x and y by $x + y$ and $x \cdot y$, respectively. (Note that the product of x and y is often denoted by xy without the use of the dot to indicate multiplication. We will not use this abridged notation in this appendix, but will within the text.) Also, by convention, we perform multiplications before additions unless parentheses are used. Although these statements are axioms, they are commonly called *laws* or *rules*. The first two of these axioms tell us that when we add or multiply two real numbers, the result is again a real number; these are the *closure laws*.

- ▶ **Closure law for addition** For all real numbers x and y , $x + y$ is a real number.
- ▶ **Closure law for multiplication** For all real numbers x and y , $x \cdot y$ is a real number.

The next two axioms tell us that when we add or multiply three real numbers, we get the same result regardless of the order of operations; these are the *associative laws*.

- ▶ **Associative law for addition** For all real numbers x , y , and z , $(x + y) + z = x + (y + z)$.
- ▶ **Associative law for multiplication** For all real numbers x , y , and z , $(x \cdot y) \cdot z = x \cdot (y \cdot z)$.

Two additional algebraic axioms tell us that the order in which we add or multiply two numbers does not matter; these are the *commutative laws*.

- ▶ **Commutative law for addition** For all real numbers x and y , $x + y = y + x$.
- ▶ **Commutative law for multiplication** For all real numbers x and y , $x \cdot y = y \cdot x$.

The next two axioms tell us that 0 and 1 are additive and multiplicative identities for the set of real numbers. That is, when we add 0 to a real number or multiply a real number by 1 we do not change this real number. These laws are called *identity laws*.

- ▶ **Additive identity law** For every real number x , $x + 0 = 0 + x = x$.
- ▶ **Multiplicative identity law** For every real number x , $x \cdot 1 = 1 \cdot x = x$.

Although it seems obvious, we also need the following axiom.

- ▶ **Identity elements axiom** The additive identity 0 and the multiplicative identity 1 are distinct, that is $0 \neq 1$.

Two additional axioms tell us that for every real number, there is a real number that can be added to this number to produce 0, and for every nonzero real number, there is a real number by which it can be multiplied to produce 1. These are the *inverse laws*.

- ▶ **Inverse law for addition** For every real number x , there exists a real number $-x$ (called the *additive inverse* of x) such that $x + (-x) = (-x) + x = 0$.
- ▶ **Inverse law for multiplication** For every nonzero real number x , there exists a real number $1/x$ (called the *multiplicative inverse* of x) such that $x \cdot (1/x) = (1/x) \cdot x = 1$.

The final algebraic axioms for real numbers are the *distributive laws*, which tell us that multiplication distributes over addition; that is, that we obtain the same result when we first add a pair of real numbers and then multiply by a third real number or when we multiply each of these two real numbers by the third real number and then add the two products.

- ▶ **Distributive laws** For all real numbers x , y , and z , $x \cdot (y + z) = x \cdot y + x \cdot z$ and $(x + y) \cdot z = x \cdot z + y \cdot z$.

ORDER AXIOMS Next, we will state the *order axioms* for the real numbers, which specify properties of the “greater than” relation, denoted by $>$, on the set of real numbers. We write $x > y$ (and $y < x$) when x is greater than y , and we write $x \geq y$ (and $y \leq x$) when $x > y$ or $x = y$. The first of these axioms tells us that given two real numbers, exactly one of three possibilities occurs: the two numbers are equal, the first is greater than the second, or the second is greater than the first. This rule is called the *trichotomy law*.

- ▶ **Trichotomy law** For all real numbers x and y , exactly one of $x = y$, $x > y$, or $y > x$ is true.

Next, we have an axiom, called *the transitivity law*, which tells us that if one number is greater than a second number and this second number is greater than a third, then the first number is greater than the third.

- ▶ **Transitivity law** For all real numbers x , y , and z , if $x > y$ and $y > z$, then $x > z$.

We also have two *compatibility laws*, which tell us that when we add a number to both sides in a greater than relationship, the greater than relationship is preserved, and when we multiply both sides of a greater than relationship by a *positive real number* (that is, a real number x with $x > 0$), the greater than relationship is preserved.

- ▶ **Additive compatibility law** For all real numbers x , y , and z , if $x > y$, then $x + z > y + z$.
- ▶ **Multiplicative compatibility law** For all real numbers x , y , and z , if $x > y$ and $z > 0$, then $x \cdot z > y \cdot z$.

We leave it to the reader (see Exercise 15) to prove that for all real numbers x , y , and z , if $x > y$ and $z < 0$, then $x \cdot z < y \cdot z$. That is, multiplication of an inequality by a negative real number reverses the direction of the inequality.

The final axiom for the set of real numbers is the *completeness property*. Before we state this axiom, we need some definitions. First, given a nonempty set A of real numbers, we say that the real number b is an **upper bound** of A if for every real number a in A , $b \geq a$. A real number s is a **least upper bound** of A if s is an upper bound of A and whenever t is an upper bound of A , then we have $s \leq t$.

- ▶ **Completeness property** Every nonempty set of real numbers that is bounded above has a least upper bound.

A1.2 Using Axioms to Prove Basic Facts

The axioms we have listed can be used to prove many properties that are often used without explicit mention. We give several examples of results we can prove using axioms and leave the proof of a variety of other properties as exercises. Although the results we will prove seem quite obvious, proving them using only the axioms we have stated can be challenging.

THEOREM 1 The additive identity element 0 of the real numbers is unique.

Proof: To show that the additive identity element 0 of the real numbers is unique, suppose that $0'$ is also an additive identity for the real numbers. This means that $0' + x = x + 0' = x$ whenever x is a real number. By the additive identity law, it follows that $0 + 0' = 0'$. Because $0'$ is an additive identity, we know that $0 + 0' = 0$. It follows that $0 = 0'$, because both equal $0 + 0'$. This shows that 0 is the unique additive identity for the real numbers. $\triangleleft$

THEOREM 2 The additive inverse of a real number x is unique.

Proof: Let x be a real number. Suppose that y and z are both additive inverses of x . Then,

$$\begin{aligned} y &= 0 + y && \text{by the additive identity law} \\ &= (z + x) + y && \text{because } z \text{ is an additive inverse of } x \\ &= z + (x + y) && \text{by the associative law for addition} \\ &= z + 0 && \text{because } y \text{ is an additive inverse of } x \\ &= z && \text{by the additive identity law.} \end{aligned}$$

It follows that $y = z$. $\triangleleft$

Theorems 1 and 2 tell us that the additive identity and additive inverses are unique. Theorems 3 and 4 tell us that the multiplicative identity and multiplicative inverses of nonzero real numbers are also unique. We leave their proofs as exercises.

THEOREM 3 The multiplicative identity element 1 of the real numbers is unique.

THEOREM 4 The multiplicative inverse of a nonzero real number x is unique.

THEOREM 5 For every real number x , $x \cdot 0 = 0$.

Proof: Suppose that x is a real number. By the additive inverse law, there is a real number y that is the additive inverse of $x \cdot 0$, so we have $x \cdot 0 + y = 0$. By the additive identity law, $0 + 0 = 0$. Using the distributive law, we see that $x \cdot 0 = x \cdot (0 + 0) = x \cdot 0 + x \cdot 0$. It follows that

$$0 = x \cdot 0 + y = (x \cdot 0 + x \cdot 0) + y.$$

Next, note that by the associative law for addition and because $x \cdot 0 + y = 0$, it follows that

$$(x \cdot 0 + x \cdot 0) + y = x \cdot 0 + (x \cdot 0 + y) = x \cdot 0 + 0.$$

Finally, by the additive identity law, we know that $x \cdot 0 + 0 = x \cdot 0$. Consequently, $x \cdot 0 = 0$. ▶

THEOREM 6

For all real numbers x and y , if $x \cdot y = 0$, then $x = 0$ or $y = 0$.

Proof: Suppose that x and y are real numbers and $x \cdot y = 0$. If $x \neq 0$, then, by the multiplicative inverse law, x has a multiplicative inverse $1/x$, such that $x \cdot (1/x) = (1/x) \cdot x = 1$. Because $x \cdot y = 0$, we have $(1/x) \cdot (x \cdot y) = (1/x) \cdot 0 = 0$ by Theorem 5. Using the associative law for multiplication, we have $((1/x) \cdot x) \cdot y = 0$. This means that $1 \cdot y = 0$. By the multiplicative identity rule, we see that $1 \cdot y = y$, so $y = 0$. Consequently, either $x = 0$ or $y = 0$. ▶

THEOREM 7

The multiplicative identity element 1 in the set of real numbers is greater than the additive identity element 0.

Proof: By the trichotomy law, either $0 = 1$, $0 > 1$, or $1 > 0$. We know by the identity elements axiom that $0 \neq 1$.

So, assume that $0 > 1$. We will show that this assumption leads to a contradiction. By the additive inverse law, 1 has an additive inverse -1 with $1 + (-1) = 0$. The additive compatibility law tells us that $0 + (-1) > 1 + (-1) = 0$; the additive identity law tells us that $0 + (-1) = -1$. Consequently, $-1 > 0$, and by the multiplicative compatibility law, $(-1) \cdot (-1) > (-1) \cdot 0$. By Theorem 5, the right-hand side of last inequality is 0. By the distributive law, $(-1) \cdot (-1) + (-1) \cdot 1 = (-1) \cdot (-1 + 1) = (-1) \cdot 0 = 0$. Hence, the left-hand side of this last inequality, $(-1) \cdot (-1)$, is the unique additive inverse of -1 , so this side of the inequality equals 1. Consequently, this last inequality becomes $1 > 0$, contradicting the trichotomy law because we had assumed that $0 > 1$.

Because we know that $0 \neq 1$ and that it is impossible for $0 > 1$, by the trichotomy law, we conclude that $1 > 0$. ▶

Links



©Hulton-Deutsch/Hulton-Deutsch Collection/Corbis via Getty Images

ARCHIMEDES (287 B.C.E.–212 B.C.E.) Archimedes was one of the greatest scientists and mathematicians of ancient times. He was born in Syracuse, a Greek city-state in Sicily. His father, Phidias, was an astronomer. Archimedes was educated in Alexandria, Egypt. After completing his studies, he returned to Syracuse, where he spent the rest of his life. Little is known about his personal life; we do not know whether he was ever married or had children. Archimedes was killed in 212 B.C.E. by a Roman soldier when the Romans overran Syracuse.

Archimedes made many important discoveries in geometry. His method for computing the area under a curve was described two thousand years before his ideas were re-invented as part of integral calculus. Archimedes also developed a method for expressing large integers inexpressible by the usual Greek method. He discovered a method for computing the volume of a sphere, as well as of other solids, and he calculated an approximation of π . Archimedes was also an accomplished engineer and inventor; his machine for pumping water, now called *Archimedes' screw*, is still in use today. Perhaps his best known discovery is the *principle of buoyancy*, which tells us that an object submerged in liquid becomes lighter by an amount equal to the weight it displaces. Some histories tell us that Archimedes was an early streaker, running naked

through the streets of Syracuse shouting “Eureka” (which means “I have found it”) when he made this discovery. He is also known for his clever use of machines that held off Roman forces sieging Syracuse for several years during the Second Punic War.

The next theorem tells us that for every real number there is an integer (where by an *integer*, we mean 0, the sum of any number of 1s, and the additive inverses of these sums) greater than this real number. This result is attributed to the Greek mathematician Archimedes. The result can be found in Book V of Euclid's *Elements*.

THEOREM 8

ARCHIMEDEAN PROPERTY For every real number x there exists an integer n such that $n > x$.

Proof: Suppose that x is a real number such that $n \leq x$ for every integer n . Then x is an upper bound of the set of integers. By the completeness property it follows that the set of integers has a least upper bound M . Because $M - 1 < M$ and M is a least upper bound of the set of integers, $M - 1$ is not an upper bound of the set of integers. This means that there is an integer n with $n > M - 1$. This implies that $n + 1 > M$, contradicting the fact that M is an upper bound of the set of integers. 

A1.3 Axioms for the Set of Positive Integers

The axioms we now list specify the set of positive integers as the subset of the set of integers satisfying four key properties. We assume the truth of these axioms in this textbook.

- ▶ **Axiom 1** The number 1 is a positive integer.
- ▶ **Axiom 2** If n is a positive integer, then $n + 1$, the *successor* of n , is also a positive integer.
- ▶ **Axiom 3** Every positive integer other than 1 is the successor of a positive integer.
- ▶ **Axiom 4 The Well-Ordering Property** Every nonempty subset of the set of positive integers has a least element.

In Sections 5.1 and 5.2 it is shown that the well-ordering principle is equivalent to the principle of mathematical induction.

- ▶ **Mathematical induction axiom** If S is a set of positive integers such that $1 \in S$ and for all positive integers n if $n \in S$, then $n + 1 \in S$, then S is the set of positive integers.

Most mathematicians take the real number system as already existing, with the real numbers satisfying the axioms we have listed in this appendix. However, mathematicians in the nineteenth century developed techniques to construct the set of real numbers, starting with more basic sets of numbers. (The process of constructing the real numbers is sometimes studied in advanced undergraduate mathematics classes. A treatment of this can be found in [Mo91], for instance.) The first step in the process is the construction of the set of positive integers using axioms 1–3 and either the well-ordering property or the mathematical induction axiom. Then, the operations of addition and multiplication of positive integers are defined. Once this has been done, the set of integers can be constructed using equivalence classes of pairs of positive integers, where $(a, b) \sim (c, d)$ if and only if $a + d = b + c$; addition and multiplication of integers can be defined using these pairs (see Exercise 21). (Equivalence relations and equivalence classes are discussed in Chapter 9.) Next, the set of rational numbers can be constructed using the equivalence classes of pairs of integers where the second integer in the pair is not zero, where $(a, b) \approx (c, d)$ if and only if $a \cdot d = b \cdot c$; addition and multiplication of rational numbers can be defined in terms of these pairs (see Exercise 22). Using infinite sequences, the set of real numbers can then be constructed from the set of rational numbers. The interested reader will find it worthwhile to read through the many details of the steps of this construction.

Exercises

Use only the axioms and theorems in this appendix in the proofs in your answers to these exercises.

1. Prove Theorem 3, which states that the multiplicative identity element of the real numbers is unique.
2. Prove Theorem 4, which states that for every nonzero real number x , the multiplicative inverse of x is unique.
3. Prove that for all real numbers x and y , $(-x) \cdot y = x \cdot (-y) = -(x \cdot y)$.
4. Prove that for all real numbers x and y , $-(x + y) = (-x) + (-y)$.
5. Prove that for all real numbers x and y , $(-x) \cdot (-y) = x \cdot y$.
6. Prove that for all real numbers x , y , and z , if $x + z = y + z$, then $x = y$.
7. Prove that for every real number x , $-(-x) = x$.
Define the **difference** $x - y$ of real numbers x and y by $x - y = x + (-y)$, where $-y$ is the additive inverse of y , and the **quotient** x/y , where $y \neq 0$, by $x/y = x \cdot (1/y)$, where $1/y$ is the multiplicative inverse of y .
8. Prove that for all real numbers x and y , $x = y$ if and only if $x - y = 0$.
9. Prove that for all real numbers x and y , $-x - y = -(x + y)$.
10. Prove that for all nonzero real numbers x and y , $1/(x/y) = y/x$, where $1/(x/y)$ is the multiplicative inverse of x/y .
11. Prove that for all real numbers w , x , y , and z , if $x \neq 0$ and $z \neq 0$, then $(w/x) + (y/z) = (w \cdot z + x \cdot y)/(x \cdot z)$.
12. Prove that for every positive real number x , $1/x$ is also a positive real number.
13. Prove that for all positive real numbers x and y , $x \cdot y$ is also a positive real number.
14. Prove that for all real numbers x and y , if $x > 0$ and $y < 0$, then $x \cdot y < 0$.

15. Prove that for all real numbers x , y , and z , if $x > y$ and $z < 0$, then $x \cdot z < y \cdot z$.
16. Prove that for every real number x , $x \neq 0$ if and only if $x^2 > 0$.
17. Prove that for all real numbers w , x , y , and z , if $w < x$ and $y < z$, then $w + y < x + z$.
18. Prove that for all positive real numbers x and y , if $x < y$, then $1/x > 1/y$.
19. Prove that for every positive real number x , there exists a positive integer n such that $n \cdot x > 1$.
- *20. Prove that between every two distinct real numbers there is a rational number (that is, a number of the form x/y , where x and y are integers with $y \neq 0$).

Exercises 21 and 22 involve the notion of an equivalence relation, discussed in Chapter 9 of the text.

- *21. Define a relation $\sim$ on the set of ordered pairs of positive integers by $(w, x) \sim (y, z)$ if and only if $w + z = x + y$. Show that the operations $[(w, x)]_{\sim} + [(y, z)]_{\sim} = [(w + y, x + z)]_{\sim}$ and $[(w, x)]_{\sim} \cdot [(y, z)]_{\sim} = [(w \cdot y + x \cdot z, x \cdot y + w \cdot z)]_{\sim}$ are well-defined, that is, they do not depend on the representative of the equivalence classes chosen for the computation.
- *22. Define a relation $\approx$ on ordered pairs of integers with second entry nonzero by $(w, x) \approx (y, z)$ if and only if $w \cdot z = x \cdot y$. Show that the operations $[(w, x)]_{\approx} + [(y, z)]_{\approx} = [(w \cdot z + x \cdot y, x \cdot z)]_{\approx}$ and $[(w, x)]_{\approx} \cdot [(y, z)]_{\approx} = [(w \cdot y, x \cdot z)]_{\approx}$ are well-defined, that is, they do not depend on the representative of the equivalence classes chosen for the computation.

2

Exponential and Logarithmic Functions

In this appendix we review some of the basic properties of exponential functions and logarithms. These properties are used throughout the text. Students requiring further review of this material should consult precalculus or calculus books, such as those mentioned in the Suggested Readings.

A2.1 Exponential Functions

Let n be a positive integer, and let b be a fixed positive real number. The function $f_b(n) = b^n$ is defined by

$$f_b(n) = b^n = b \cdot b \cdot b \cdot \cdots \cdot b,$$

where there are n factors of b multiplied together on the right-hand side of the equation.

We can define the function $f_b(x) = b^x$ for all real numbers x using techniques from calculus. The function $f_b(x) = b^x$ is called the **exponential function to the base b** . We will not discuss how to find the values of exponential functions to the base b when x is not an integer.

Two of the important properties satisfied by exponential functions are given in Theorem 1. Proofs of these and other related properties can be found in calculus texts.

THEOREM 1

Let b be a positive real number and x and y real numbers. Then

1. $b^{x+y} = b^x b^y$, and
2. $(b^x)^y = b^{xy}$.

We display the graphs of some exponential functions in Figure 1.

A2.2 Logarithmic Functions

Suppose that b is a real number with $b > 1$. Then the exponential function b^x is strictly increasing (a fact shown in calculus). It is a one-to-one correspondence from the set of real numbers to the set of nonnegative real numbers. Hence, this function has an inverse $\log_b x$, called the **logarithmic function to the base b** . In other words, if b is a real number greater than 1 and x is a positive real number, then

$$b^{\log_b x} = x.$$

The value of this function at x is called the **logarithm of x to the base b** .

From the definition, it follows that

$$\log_b b^x = x.$$

We give several important properties of logarithms in Theorem 2.

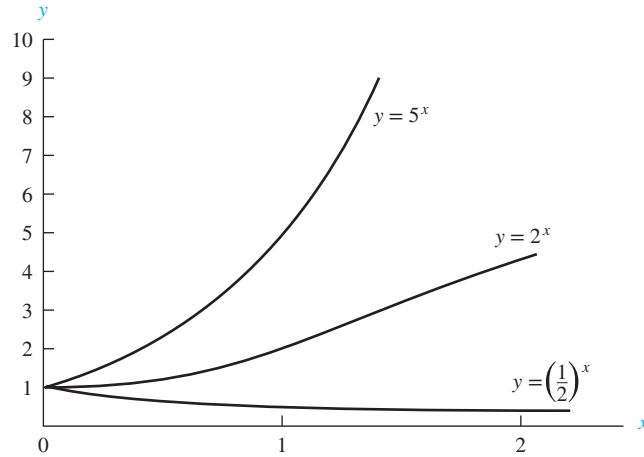


FIGURE 1 Graphs of the exponential functions to the bases $\frac{1}{2}$, 2, and 5.

THEOREM 2

Let b be a real number greater than 1. Then

1. $\log_b(xy) = \log_b x + \log_b y$ whenever x and y are positive real numbers, and
2. $\log_b(x^y) = y \log_b x$ whenever x is a positive real number and y is a real number.

Proof: Because $\log_b(xy)$ is the unique real number with $b^{\log_b(xy)} = xy$, to prove part 1 it suffices to show that $b^{\log_b x + \log_b y} = xy$. By part 1 of Theorem 1, we have

$$\begin{aligned} b^{\log_b x + \log_b y} &= b^{\log_b x} b^{\log_b y} \\ &= xy. \end{aligned}$$

To prove part 2, it suffices to show that $b^{y \log_b x} = x^y$. By part 2 of Theorem 1, we have

$$\begin{aligned} b^{y \log_b x} &= (b^{\log_b x})^y \\ &= x^y. \end{aligned}$$



The following theorem relates logarithms to two different bases.

THEOREM 3

CHANGE OF BASE FORMULA FOR LOGARITHMS Let a and b be real numbers greater than 1, and let x be a positive real number. Then

$$\log_a x = \log_b x / \log_b a.$$

Let a and b be real numbers

Proof: To prove this result, it suffices to show that

$$b^{\log_a x \cdot \log_b a} = x.$$

By part 2 of Theorem 1, we have

$$\begin{aligned} b^{\log_a x \cdot \log_b a} &= (b^{\log_b a})^{\log_a x} \\ &= a^{\log_a x} \\ &= x. \end{aligned}$$

This completes the proof.



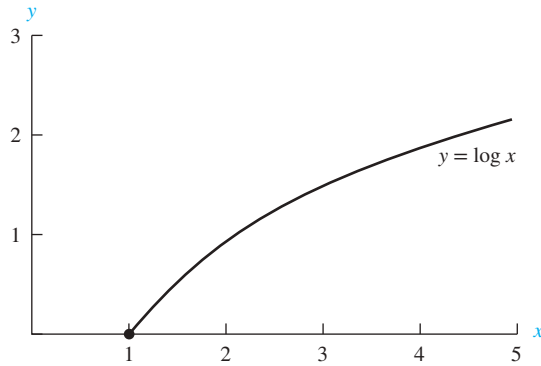


FIGURE 2 The graph of $f(x) = \log x$.

Because the base used most often for logarithms in this text is $b = 2$, the notation $\log x$ is used throughout the text to denote $\log_2 x$.

The graph of the function $f(x) = \log x$ is displayed in Figure 2. From Theorem 3, when a base b other than 2 is used, a function that is a constant multiple of the function $\log x$, namely, $(1/\log b) \log x$, is obtained.

Exercises

- Express each of the following quantities as powers of 2.
 a) $2 \cdot 2^2$ b) $(2^2)^3$ c) $2^{(2^2)}$
- Find each of the following quantities.
 a) $\log_2 1024$ b) $\log_2 1/4$ c) $\log_4 8$
- Suppose that $\log_4 x = y$, where x is a positive real number. Find each of the following quantities.
 a) $\log_2 x$ b) $\log_8 x$ c) $\log_{16} x$
- Let a , b , and c be positive real numbers. Show that $a^{\log_b c} = c^{\log_b a}$.
- Draw the graph of $f(x) = b^x$ for all real numbers x if b is
 a) 3. b) $1/3$. c) 1.
- Draw the graph of $f(x) = \log_b x$ for positive real numbers x if b is
 a) 4. b) 100. c) 1000.

3

Pseudocode

Links 

The algorithms in this text are described both in English and in **pseudocode**. Pseudocode is an intermediate step between an English language description of the steps of a procedure and a specification of this procedure using an actual programming language. The advantages of using pseudocode include the simplicity with which it can be written and understood and the ease of producing actual computer code (in a variety of programming languages) from the pseudocode. We will describe the particular types of **statements**, or high-level instructions, of the pseudocode that we will use. Each of these statements in pseudocode can be translated into one or more statements in a particular programming language, which in turn can be translated into one or more (possibly many) low-level instructions for a computer.

This appendix describes the format and syntax of the pseudocode used in the text. This pseudocode is designed so that its basic structure resembles that of commonly used programming languages, such as C++ and Java, which are currently the most commonly taught programming languages. However, the pseudocode we use will be a lot looser than a formal programming language because a lot of English language descriptions of steps will be allowed.

This appendix is not meant for formal study. Rather, it should serve as a reference guide for students when they study the descriptions of algorithms given in the text and when they write pseudocode solutions to exercises.

A3.1 Procedure Statements

The pseudocode for an algorithm begins with a **procedure** statement that gives the name of an algorithm, lists the input variables, and describes what kind of variable each input is. For instance, the statement

```
procedure maximum(L: list of integers)
```

is the first statement in the pseudocode description of the algorithm, which we have named *maximum*, that finds the maximum of a list *L* of integers.

A3.2 Assignments and Other Types of Statements

An assignment statement is used to assign values to variables. In an assignment statement the left-hand side is the name of the variable and the right-hand side is an expression that involves constants, variables that have been assigned values, or functions defined by procedures. The right-hand side may contain any of the usual arithmetic operations. However, in the pseudocode in this book it may include any well-defined operation, even if this operation can be carried out only by using a large number of statements in an actual programming language.

The symbol `:=` is used for assignments. Thus, an assignment statement has the form

```
variable := expression
```

For example, the statement

```
max := a
```

assigns the value of *a* to the variable *max*. A statement such as

```
x := largest integer in the list L
```

can also be used. This sets *x* equal to the largest integer in the list *L*. To translate this statement into an actual programming language would require more than one statement. Also, the instruction

```
interchange a and b
```

can be used to interchange *a* and *b*. We could also express this one statement with several assignment statements (see Exercise 2), but for simplicity, we will often prefer this abbreviated form of pseudocode.

A3.3 Comments

In the pseudocode in this book, statements enclosed in curly braces are not executed. Such statements serve as comments or reminders that help explain how the procedure works. For instance, the statement

```
{x is the largest element in L}
```

can be used to remind the reader that at that point in the procedure the variable *x* equals the largest element in the list *L*.

A3.4 Conditional Constructions

The simplest form of the conditional construction that we will use is

```
if condition then statement
```

or

```
if condition then  
    block of statements
```

Here, the condition is checked, and if it is true, then the statement or block of statements given is carried out. In particular, the pseudocode

```
if condition then
    statement 1
    statement 2
    statement 3
    .
    .
    .
    statement  $n$ 
```

tells us that the statements in the block are executed sequentially if the condition is true.

For example, in Algorithm 1 in Section 3.1, which finds the maximum of a set of integers, we use a conditional statement to check whether $max < a_i$ for each variable; if it is, we assign the value of a_i to max .

Often, we require the use of a more general type of construction. This is used when we wish to do one thing when the indicated condition is true, but another when it is false. We use the construction

```
if condition then statement 1
else statement 2
```

Note that either one or both of statement 1 and statement 2 can be replaced with a block of statements.

Sometimes, we require the use of an even more general form of a conditional. The general form of the conditional construction that we will use is

```
if condition 1 then statement 1
else if condition 2 then statement 2
else if condition 3 then statement 3
    .
    .
    .
else if condition  $n$  then statement  $n$ 
else statement  $n + 1$ 
```

When this construction is used, if condition 1 is true, then statement 1 is carried out, and the program exits this construction. In addition, if condition 1 is false, the program checks whether condition 2 is true; if it is, statement 2 is carried out, and so on. Thus, if none of the first $n - 1$ conditions hold, but condition n does, statement n is carried out. Finally, if none of condition 1, condition 2, condition 3, ..., condition n is true, then statement $n + 1$ is executed. Note that any of the $n + 1$ statements can be replaced by a block of statements.

A3.5 Loop Constructions

There are two types of loop construction in the pseudocode in this book. The first is the “for” construction, which has the form

```
for variable := initial value to final value
    statement
```

or

```
for variable := initial value to final value
    block of statements
```

where *initial value* and *final value* are integers. Here, at the start of the loop, *variable* is assigned *initial value* if *initial value* is less than or equal to *final value*, and the statements at the end of this construction are carried out with this value of *variable*. Then *variable* is increased by one, and the statement, or the statements in the block, are carried out with this new value of *variable*. This is repeated until *variable* reaches *final value*. After the instructions are carried out with *variable* equal to *final value*, the algorithm proceeds to the next statement. When *initial value* exceeds *final value*, none of the statements in the loop is executed.

We can use the “for” loop construction to find the sum of the positive integers from 1 to n with the following pseudocode.

```
sum := 0
for i := 1 to n
    sum := sum + i
```

Also, the more general “for” statement, of the form

```
for all elements with a certain property
```

is used in this text. This means that the statement or block of statements that follow are carried out successively for the elements with the given property.

The second type of loop construction that we will use is the “while” construction. This has the form

```
while condition
    statement
```

or

```
while condition
    block of statements
```

When this construction is used, the condition given is checked, and if it is true, the statements that follow are carried out, which may change the values of the variables that are part of the

condition. If the condition is still true after these instructions have been carried out, the instructions are carried out again. This is repeated until the condition becomes false. As an example, we can find the sum of the integers from 1 to n using the following block of pseudocode including a “while” construction.

```

sum := 0
while n > 0
    sum := sum + n
    n := n - 1

```

Note that any “for” construction can be turned into a “while” construction (see Exercise 3). However, it is often easier to understand the “for” construction. So, when it makes sense, we will use the “for” construction in preference to the corresponding “while” construction.

A3.6 Loops within Loops

Loops or conditional statements are often used within other loops or conditional statements. In the pseudocode used in this book, we use successive levels of indentation to indicate nested loops, which are loops within loops, and which blocks of commands correspond to which loops.

A3.7 Using Procedures in Other Procedures

We can use a procedure from within another procedure (or within itself in a recursive program) simply by writing the name of this procedure followed by the inputs to this procedure. For instance,

```

max( $L$ )

```

will carry out the procedure *max* with the input list L . After all the steps of this procedure have been carried out, execution carries on with the next statement in the procedure.

A3.8 Return Statements

We use a **return** statement to show where a procedure produces output. A return statement of the form

```

return  $x$ 

```

produces the current value of x as output. The output x can involve the value of one or more functions, including the same function under evaluation, but at a smaller value. For instance, the statement

```

return  $f(n - 1)$ 

```

is used to call the algorithm with input of $n - 1$. This means that the algorithm is run again with input equal to $n - 1$.

Exercises

1. What is the difference between the following blocks of two assignment statements?

$a := b$

$b := c$

and

$b := c$

$a := b$

2. Give a procedure using assignment statements to interchange the values of the variables x and y . What is the minimum number of assignment statements needed to do this?

3. Show how a loop of the form

for $i := \textit{initial value}$ **to** $\textit{final value}$
statement

can be written using the “while” construction.